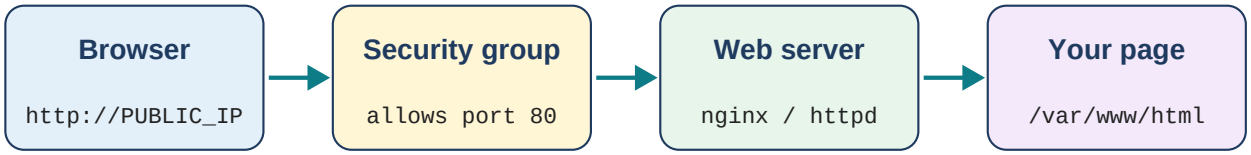
Deploy Web Servers on EC2: NGINX and Apache

Install, start and verify a website on a Linux instance. Simple notes with steps, commands and outputs

Part	Topic	One-line idea
1	What is a web server?	Software that sends web pages to browsers
2	Deploy NGINX (Ubuntu)	apt install, start, host your page
3	Deploy Apache HTTPD (Amazon Linux / Red Hat)	yum install, start, host your page
4	NGINX vs Apache	Same job, different details
5	Troubleshooting	Why does my page not load?
6	Recap: Windows instance and remote access	From the previous lesson
7	Quick quiz	Revise with answers

1. What is a Web Server?

A **web server** is software that **waits for requests from browsers and sends back web pages**. Install it on your EC2 instance, put your website files in a folder, and anyone allowed by the security group can open your site using the instance's public IP.



Term	Simple meaning
NGINX	A fast, lightweight web server and reverse proxy. Pronounced 'engine-x'.
HTTPD (Apache)	A very popular open-source web server, known for flexibility. The service is named **httpd**.
Reverse proxy	A server in front of other servers: it receives requests and passes them on (and can share the load).
Document root	The folder with your website files. Usually **/var/www/html**.
index.html	The default page shown when you open the site.
Port 80 / 443	HTTP (plain) / HTTPS (encrypted).
Service	A program run and controlled by systemd with systemctl (start, stop, enable).

Before you start

- An **EC2 Linux instance** that is running, and you can log in with SSH.

- The **security group** allows inbound **HTTP (80)** and **HTTPS (443)** from Anywhere (0.0.0.0/0). Websites are meant to be public, unlike SSH.
- Ubuntu uses **apt**. Amazon Linux and Red Hat use **yum** (or dnf).

Remember: The **security group is the main firewall**. If port 80 is not allowed there, nothing you do inside the server will make the page open.

2. Deploy NGINX (Ubuntu)

1. **Update the package list**
   sudo apt update
2. **Install NGINX**
   sudo apt install nginx -y
3. **Start and enable the service**
   systemctl start (now) and enable (at every boot)
4. **Verify**
   Browser: http://PUBLIC_IP shows the NGINX welcome page
5. **Firewall (only if ufw is on)**
   sudo ufw allow 'Nginx Full'
6. **Host your own page**
   Replace /var/www/html/index.html
7. **Restart or reload, then verify**
   Open the public IP again to see your site

Step 1 to 3: install and start

COPY AND PASTE: Install NGINX

```
sudo apt update
sudo apt install nginx -y
sudo systemctl start nginx
sudo systemctl enable nginx
systemctl is-active nginx
systemctl is-enabled nginx
```

Command	What it does (in simple words)
`sudo apt update`	Refreshes the list of available software.
`sudo apt install nginx -y`	Installs NGINX. -y answers yes automatically.
`sudo systemctl start nginx`	Starts the web server now. (On Ubuntu it usually starts by itself after install.)

Command	What it does (in simple words)
`sudo systemctl enable nginx`	Makes it start automatically every time the server boots.
`systemctl is-active nginx`	Prints active when it is running.
`systemctl is-enabled nginx`	Prints enabled when it starts at boot.

Output (what students will see)

```
active
enabled
```

Step 4: verify

COPY AND PASTE: Check from inside the server, then from your browser

```
curl -sI http://localhost | head -3
sudo ss -tlnp | grep ':80'
curl -s ifconfig.me; echo
```

Command	What it does (in simple words)
`curl -sI http://localhost | head -3`	Asks the server itself for the page headers. 200 OK means it works.
`sudo ss -tlnp | grep ':80'`	Shows which program is listening on port 80.
`curl -s ifconfig.me`	Shows the server's public IP (use it in your browser).

Output (what students will see)

```
HTTP/1.1 200 OK                        <-- the web server answers
Server: nginx/1.18.0 (Ubuntu)          <-- it is NGINX
LISTEN 0 511 0.0.0.0:80 ... nginx      <-- nginx listens on port 80
13.233.10.25                           <-- open http://13.233.10.25 in a browser
```

Browser tip: In the browser type **http://** and the public IP. Some browsers try **https** first and show an error, because HTTPS is not set up yet. You should see **Welcome to nginx!**.

Step 5: firewall (only if ufw is enabled)

On a new EC2 Ubuntu instance the OS firewall **ufw is normally off**, so skip this. If it is on, allow the web traffic:

COPY AND PASTE: ufw: check and allow web traffic

```
sudo ufw status
sudo ufw allow 'Nginx Full'
sudo ufw status
```

Command	What it does (in simple words)
`sudo ufw status`	Shows Status: inactive (off) or active with rules.

Command	What it does (in simple words)
`sudo ufw allow 'Nginx Full'`	Opens ports 80 and 443 (an NGINX profile).

> **Warning:** Never run `sudo ufw enable` before allowing SSH with `sudo ufw allow OpenSSH`. Otherwise you lock yourself out of the server.

Step 6 and 7: host your own website

The default NGINX page is replaced by creating **index.html** in **/var/www/html**. The block below writes the file with one paste, so you do not need an editor.

COPY AND PASTE: Create a custom website

```
sudo tee /var/www/html/index.html > /dev/null << 'EOF'
<!DOCTYPE html>
<html>
<head><title>My First EC2 Website</title></head>
<body>
  <h1>Hello from my EC2 server!</h1>
  <p>This page is served by NGINX.</p>
</body>
</html>
EOF
sudo nginx -t
sudo systemctl restart nginx
curl -s http://localhost
```

Command	What it does (in simple words)
`sudo tee /var/www/html/index.html > /dev/null << 'EOF' ...`	Writes the lines up to EOF into the file as root. (sudo echo > file would fail, because the shell opens the file before sudo runs. tee fixes that.)
`sudo nginx -t`	Tests the NGINX configuration for mistakes.
`sudo systemctl restart nginx`	Restarts NGINX (reload is gentler: it keeps connections).
`curl -s http://localhost`	Shows the page text: your HTML.

Output (what students will see)

```
nginx: configuration file /etc/nginx/nginx.conf test is successful   <-- config OK
<h1>Hello from my EC2 server!</h1>                                   <-- your page
```

Using an editor instead: `sudo nano /var/www/html/index.html`, type your HTML, save with **Ctrl+O**, **Enter**, and exit with **Ctrl+X**. Reload the browser page to see the change.

Useful NGINX commands

Command	Use
`sudo systemctl status nginx`	Is it running? Shows recent log lines.
`sudo systemctl stop nginx`	Stop the web server.

Command	Use
`sudo systemctl reload nginx`	Apply config changes without dropping connections.
`sudo nginx -t`	Test the configuration before reloading.
`sudo tail -n 5 /var/log/nginx/access.log`	Last 5 requests (who opened your site).
`sudo tail -n 5 /var/log/nginx/error.log`	Last 5 errors.

3. Deploy Apache HTTPD (Amazon Linux / Red Hat)

1. **Update the system**
   sudo yum update -y
2. **Install HTTPD**
   sudo yum install httpd -y
3. **Start and enable the service**
   systemctl start httpd, then enable httpd
4. **Verify**
   Browser: http://PUBLIC_IP shows the Apache test page
5. **Firewall (only if firewalld is running)**
   firewall-cmd --permanent --add-service=http / https, then --reload
6. **Host your own page**
   Create /var/www/html/index.html
7. **Restart, then verify**
   Open the public IP again to see your site

Different names: This lesson uses **yum** and **httpd**, which are the Amazon Linux and Red Hat names. Log in as **ec2-user** on Amazon Linux. On **Ubuntu** the same server is called **apache2**: `sudo apt install apache2 -y` and `sudo systemctl start apache2`.

Step 1 to 3: install and start

COPY AND PASTE: Install HTTPD

```
sudo yum update -y
sudo yum install httpd -y
sudo systemctl start httpd
sudo systemctl enable httpd
systemctl is-active httpd
systemctl is-enabled httpd
```

Command	What it does (in simple words)
sudo yum update -y	Updates installed software. -y answers yes.
sudo yum install httpd -y	Installs the Apache web server.
sudo systemctl start httpd	Starts it now. (It does not start by itself after install.)
sudo systemctl enable httpd	Starts it at every boot.
systemctl is-active / is-enabled httpd	Should print active and enabled.

Output (what students will see)

```
active
enabled
```

Step 4: verify

COPY AND PASTE: Check from inside the server

```
curl -sI http://localhost | head -3
sudo ss -tlnp | grep ':80'
```

Output (what students will see)

```
HTTP/1.1 403 Forbidden  or  200 OK     <-- see the note below
Server: Apache/2.4 (Amazon Linux)
LISTEN 0 511 *:80 ... httpd             <-- httpd listens on port 80
```

Note: A fresh Apache with **no index.html** shows its default test page (some systems answer **403 Forbidden** on the headers check). This is normal. After you add your own **index.html** it answers **200 OK**.

Step 5: firewall (only if firewalld is running)

COPY AND PASTE: firewalld: allow web traffic

```
sudo firewall-cmd --state
sudo firewall-cmd --permanent --add-service=http
sudo firewall-cmd --permanent --add-service=https
sudo firewall-cmd --reload
```

Command	What it does (in simple words)
sudo firewall-cmd --state	Prints running if firewalld is active. (Amazon Linux usually does not run it: skip the rest.)
--permanent --add-service=http / https	Adds ports 80 and 443 to the permanent rules.
sudo firewall-cmd --reload	Applies the new rules.

Step 6 and 7: host your own website

COPY AND PASTE: Create a custom website

```
cd /var/www/html
sudo tee index.html > /dev/null << 'EOF'
<!DOCTYPE html>
<html>
<head><title>My Apache Website</title></head>
<body>
  <h1>Hello from Apache on EC2!</h1>
  <p>This page is served by httpd.</p>
</body>
</html>
EOF
sudo apachectl configtest
sudo systemctl restart httpd
curl -s http://localhost
```

Command	What it does (in simple words)
`cd /var/www/html`	Go to the document root.
`sudo tee index.html > /dev/null << 'EOF' ...`	Create index.html with your HTML (one paste). Or use sudo nano index.html.
`sudo apachectl configtest`	Tests the Apache configuration. Should print Syntax OK.
`sudo systemctl restart httpd`	Restarts Apache.
`curl -s http://localhost`	Shows your HTML.

Output (what students will see)

```
Syntax OK                          <-- configuration is fine
<h1>Hello from Apache on EC2!</h1>      <-- your page
```

Useful HTTPD commands

Command	Use
`sudo systemctl status httpd`	Is it running?
`sudo systemctl reload httpd`	Apply config changes gently.
`sudo apachectl configtest`	Check the configuration.
`sudo tail -n 5 /var/log/httpd/access_log`	Last 5 requests.
`sudo tail -n 5 /var/log/httpd/error_log`	Last 5 errors.

4. NGINX vs Apache HTTPD

	NGINX	Apache HTTPD
Known for	Speed, low memory, handling many connections, reverse proxy and load balancing	Flexibility, many modules, easy per-folder settings (.htaccess)
Style	Event-driven	Process or thread based
Service name	nginx	httpd (Amazon Linux, Red Hat) / apache2 (Ubuntu)
Install	apt install nginx or yum install nginx	yum install httpd or apt install apache2
Document root	/var/www/html (Ubuntu)	/var/www/html
Config	/etc/nginx/nginx.conf	/etc/httpd/conf/httpd.conf (Ubuntu: /etc/apache2)
Logs	/var/log/nginx/	/var/log/httpd/ (Ubuntu: /var/log/apache2/)
Test config	nginx -t	apachectl configtest

Remember: Both do the same job: serve web pages on port 80. **Only one program can use port 80 at a time**. If you install both and start both, the second one fails with **Address already in use**.

5. Troubleshooting: My Page Does Not Load

What you see	Likely cause	Fix
Browser keeps loading, then times out	Security group does not allow port 80	Add inbound HTTP 80 from 0.0.0.0/0
Connection refused	Web server is not running	sudo systemctl start nginx (or httpd)
Wrong or old page	No new index.html in /var/www/html, or the browser cache	Check the file, hard-refresh the browser (Ctrl+F5)
403 Forbidden	Missing index.html or the file is not readable	Create index.html; check permissions (644)
Address already in use	Another service uses port 80	sudo ss -tlnp | grep ':80', stop the other one
Config error after editing	Typing mistake in the config	Run nginx -t or apachectl configtest
Works on the server, not from outside	Security group, or the OS firewall	Check the security group, then ufw or firewalld

COPY AND PASTE: Quick health check (NGINX)

```
sudo systemctl status nginx --no-pager | head -5
sudo nginx -t
sudo tail -n 3 /var/log/nginx/error.log
ls -l /var/www/html
```

Command	What it does (in simple words)
`sudo systemctl status nginx --no-pager | head -5`	Running or failed? --no-pager stops the output from pausing.
`sudo nginx -t`	Is the configuration valid?
`sudo tail -n 3 /var/log/nginx/error.log`	The last three error messages.
`ls -l /var/www/html`	Is index.html there, and is it readable (-rw-r--r--)?

Next steps: Later you can point a **domain name** (DNS A record) to the instance's public or **Elastic IP**, and add a free certificate (Let's Encrypt) to use **HTTPS** on port 443. See the networking notes for DNS.

6. Recap: Windows Instance and Remote Access

Step to launch a Windows instance	Notes
1. Open EC2, Launch instance	Check the region first.
2. Choose a Windows Server AMI	Windows Server 2019 Base in the notes (newer versions are also offered).
3. Choose instance type	t2.micro if Free tier eligible. It can feel slow with 1 GiB RAM.
4. Storage	Keep the default or increase.
5. Security group	Inbound RDP: TCP 3389 from My IP.
6. Launch and download the .pem key	Needed to decrypt the password.
7. Get the Windows password	Instance, Connect, RDP client, Get password, upload .pem, Decrypt.
8. Connect	Remote Desktop: public IP, user Administrator, the decrypted password.

	Linux instance	Windows instance
Protocol and port	SSH, 22	RDP, 3389
Command or client	ssh -i key.pem ubuntu@PUBLIC_IP	Remote Desktop Connection / RDP client
Login	ubuntu, ec2-user ... with the .pem key	Administrator with the decrypted password
Troubleshoot	Instance running, your IP allowed on port 22	Port 3389 allowed, instance has a public IP

7. Quick Quiz and Takeaways

Question	Answer
What does a web server do?	Receives browser requests and sends back web pages.
Which security group ports are needed for a website?	80 (HTTP) and 443 (HTTPS).
Install NGINX on Ubuntu?	sudo apt update, then sudo apt install nginx -y.
Install Apache on Amazon Linux?	sudo yum install httpd -y.
What do start and enable do?	start runs it now. enable runs it at every boot.
Where is the website folder?	/var/www/html (file: index.html).
How do you test the config?	sudo nginx -t or sudo apachectl configtest.
Page times out from outside, but curl localhost works. Why?	The security group does not allow port 80.
Can NGINX and Apache both listen on port 80?	No. Only one program can use a port at a time.

- NGINX and Apache HTTPD are strong platforms for hosting web applications.
- Install, start, enable, verify, then replace index.html with your own page.
- Open ports 80 and 443 in the security group, and check the OS firewall only if it is on.